

CareerLaunch

Step-by-Step Build Guide

How to create every part of the Cohort Enrolment & Student Success application: objects and fields, validation rules, flows, Apex, Lightning Web Components, the Enrolment Console page, the Agentforce agent, the permission set, tests and deployment.

GitHub: github.com/dangutdavid/oxfordabledeveloper1_training

Target org: maren.dangut@gmail.com · **Version:** 1.0 · August 2026

Already done for you: everything in this guide has been built, deployed to the org maren.dangut@gmail.com, tested (10/10 Apex tests passing) and pushed to the GitHub repository above. Use this guide either to **rebuild each piece by hand** (the best way to learn) or to **understand what was deployed**. Sections 8–9 (page activation and the Agentforce agent) are manual steps you must finish in the org.

Contents

Section	What you build
1. Prerequisites	Org, CLI, VS Code, Git
2. The Course object	Custom object, 5 fields, tab (Lab 1)
3. The Cohort object	Object, lookup, roll-up, 2 formulas, validation rule (Lab 2)
4. The Enrolment object	Master-detail, unique external ID, validation rule (Lab 2)
5. Sample data	Courses, cohorts, students
6. Flows	Record-triggered, screen, scheduled (Lab 3)
7. Apex	Trigger, handler, Queueable, invocables, controllers (Lab 4)
8. Lightning Web Components	cohortExplorer, enrolmentPanel, enrolmentConsole + app page (Lab 5)
9. Agentforce agent	Actions, topic, instructions, testing (Lab 6)
10. Testing	Test data factory, test classes, coverage (Lab 7)
11. Security & deployment	Permission set, Git, CLI deploy, post-deploy checklist (Lab 8)
12. UAT script	Prove every user story works

1 · Prerequisites

- 1.1** A Salesforce Developer Edition org (free, never expires): developer.salesforce.com/signup.
- 1.2** Google Chrome, Visual Studio Code + the **Salesforce Extension Pack**.
- 1.3** Salesforce CLI — confirm with `sf --version` in a terminal.
- 1.4** Git installed and a GitHub account.

5. **1.5** In the org: Setup → search **Deliverability** → set Access level to **All Email** (needed for the welcome email in Section 6).

Get the finished source: `git clone https://github.com/dangutdavid/oxfodablenameveloper1_training.git`

2 · The Course object (Lab 1)

2.1 Create the object

1. Log in → gear icon (top right) → **Setup** → **Object Manager** tab → **Create** ▼ → **Custom Object**.
2. Enter the values below, then tick **Launch New Custom Tab Wizard after saving** and click **Save**. In the tab wizard pick any tab style → Next → Next → Save.

Setting	Value
Label / Plural Label	Course / Courses
Object Name	Course
Record Name / Data Type	Course Name / Text
Tick	Allow Reports · Allow Activities · Track Field History

2.2 Add the fields

Object Manager → Course → **Fields & Relationships** → **New**. For each field: choose the type → enter details → leave field-level security defaults (Next) → leave page layout ticked (Save & New).

Field Label	Data Type	Settings	API name created
Course Level	Picklist	Values, one per line: Introductory, Advanced	Course_Level__c
Duration (Weeks)	Number	Length 2, Decimal Places 0	Duration_Weeks__c
Fee	Currency	Length 10, Decimal Places 2	Fee__c
Active	Checkbox	Default: Checked	Active__c
Description	Text Area (Long)	Length 1000	Description__c

3 · The Cohort object (Lab 2)

3.1 Create the object

Setup → Object Manager → Create ▼ → Custom Object:

Setting	Value
Label / Plural	Cohort / Cohorts
Record Name	Cohort Number — Data Type Auto Number , Display Format <code>C0-{0000}</code> , Starting Number 1
Tick	Allow Reports · Allow Activities · Launch New Custom Tab Wizard

3.2 Add the basic fields (leave the calculated ones for 3.3)

Field	Type	Settings
Course	Lookup Relationship	Related To: Course ; tick <i>Required</i>
Start Date	Date	Tick <i>Required</i>
End Date	Date	—
Delivery Mode	Picklist	Online, In-Person, Hybrid
Capacity	Number	Length 3, Decimals 0, Default Value 20, tick <i>Required</i>
Status	Picklist	Planned, Open, Full, Running, Completed, Cancelled

Do not build Enrolled Count yet — a roll-up summary is only possible after the master-detail from Enrolment exists (Section 4). Come back to step 3.3 after finishing Section 4.

3.3 Add the three calculated fields (after Section 4)

Field	Type	Definition
Enrolled Count	Roll-Up Summary	Summarised Object: Enrolments · Roll-Up Type: COUNT · Filter: Status not equal to Cancelled
Seats Remaining	Formula → Number (0 dp)	<code>Capacity__c - Enrolled_Count__c</code>
Cohort Label	Formula → Text	<code>Course__r.Name & " - " & TEXT(Start_Date__c)</code>

3.4 Validation rule

Object Manager → Cohort → **Validation Rules** → New. Name: `End_Date_After_Start_Date`

```
AND(
  NOT(ISBLANK(End_Date__c)),
  End_Date__c < Start_Date__c
)
```

Error message: *End Date cannot be earlier than Start Date.* · Error location: End Date.

4 · The Enrolment object (Lab 2)

Spelling matters: it is **Enrolment** with one “l” (British spelling) everywhere, or the Apex will not compile.

4.1 Create the object

Setting	Value
Label / Plural	Enrolment / Enrolments
Record Name	Enrolment Number — Auto Number, Format <code>ENR-{00000}</code>
Tick	Allow Reports · Allow Activities · Launch New Custom Tab Wizard

4.2 Add the fields — in this order (the master-detail must be first)

Field	Type	Settings
Cohort	Master-Detail Relationship	Related To: Cohort . Accept sharing defaults.
Student	Lookup Relationship	Related To: Contact ; tick <i>Required</i>
Status	Picklist	Applied (set as default), Confirmed, Waitlisted, Cancelled, Completed
Enrolment Date	Date	—
Amount Paid	Currency	Length 10, Decimals 2, Default 0
Enrolment Key	Text	Length 64; tick External ID and Unique (case-insensitive)
Attendance %	Percent	Length 3, Decimals 0
Certificate Issued	Checkbox	Default: Unchecked

4.3 Validation rule

Name: `Payment_Not_Above_Fee` — a cross-object formula spanning two relationships:

```
Amount_Paid__c > Cohort__r.Course__r.Fee__c
```

Error message: *Amount Paid cannot exceed the course fee.*

4.4 Contact field

Object Manager → **Contact** → Fields & Relationships → New → Number, Length 3, Decimals 0, Label **Total Enrolments** (`Total_Enrolments__c`). Maintained automatically by the Queueable job (Section 7).

Now go back to step 3.3 and create the roll-up and formulas on Cohort. Then tidy the Cohort page layout: add Enrolled Count, Seats Remaining, Cohort Label, and confirm the Enrolments related list is present.

5 · Sample data

Either create records by hand (Courses tab → New, etc.) or run the seed script from the repo:

```
sf apex run --file scripts/apex/seed.apex --target-org <your-org-alias>
```

Object	Records
Course	Salesforce Developer – Introductory (3 wks, £500) · Salesforce Developer – Advanced (10 wks, £1500)
Cohort	CO-0001: Introductory, starts today+14, Capacity 3 , Open · CO-0002: Advanced, starts today+30, Capacity 10, Open
Contact	3 students with real-looking email addresses (the agent searches on email)

Checkpoint (US-03): create one Enrolment linking a Contact to CO-0001, then open CO-0001 and confirm **Enrolled Count = 1** and **Seats Remaining = 2** — with no automation at all.

6 · Flows (Lab 3)

6.1 Record-triggered flow — Enrolment_Confirmed_Actions (US-06, US-07)

Setup → search **Flows** → New Flow → **Record-Triggered Flow**:

Setting	Value
Object	Enrolment
Trigger	A record is created or updated
Entry conditions	Status__c Equals Confirmed — and “Only when a record is updated to meet the condition requirements”
Optimise for	Actions and Related Records (after-save — we touch other records)

Add these elements in order:

1. **Get Records** “Get_Cohort”: object Cohort, Id Equals `{!$Record.Cohort__c}`, only first record, fields: Seats_Remaining__c, Status__c, Start_Date__c.
2. **Action** → **Send Email**: Recipient `{!$Record.Student__r.Email}` · Subject Your place is confirmed — `{!$Record.Cohort__r.Cohort_Label__c}` · Body: a text template with the welcome message and start date. **Add a Fault path from this action to the next element** so a failed email never blocks the enrolment save.
3. **Create Records** — a Task: Subject Send welcome pack to `{!$Record.Student__r.Name}`, WhatId `{!$Record.Id}`, OwnerId `{!$Record.Cohort__r.OwnerId}` (the cohort owner — Enrolment is master-detail so it has no owner of its own), ActivityDate `{!$Flow.CurrentDate}`, Priority High.
4. **Decision** “Is Cohort Now Full?": Yes when `{!Get_Cohort.Seats_Remaining__c} <= 0`.
5. **Update Records** on the Yes path: update Cohort where Id = `{!$Record.Cohort__c}`, set Status__c = Full.
6. **Save** as “Enrolment Confirmed Actions”, then **Activate**.

Test it: change an Enrolment’s Status to Confirmed → a Task appears on the record and filling CO-0001’s last seat flips the Cohort Status to Full (UAT-06, UAT-07).

6.2 Screen flow — Quick_Enrol_Student (US-08)

Setup → Flows → New Flow → **Screen Flow**:

Element	Content
Screen 1 “Which course?”	Picklist using a record choice set on Course where <code>Active = true</code> (label: Name, value: Id)
Screen 2 “Which cohort?”	Radio buttons using a record choice set on Cohort where <code>Course = {!Course_Picklist}</code> AND <code>Status = Open</code> , label Cohort Label , sorted by Start Date
Screen 3 “Which student?”	Lookup screen component: Object API Name <code>Enrolment__c</code> , Field API Name <code>Student__c</code>
Get Records	Get the selected Course record (to pass its Name to the Apex action)
Action	Apex Action → Find Available Cohorts (CohortAvailabilityService) with Course Name — re-checks seats before writing
Decision	<code>{!Check_Seats.seatsAvailable} = true ?</code>
Create Records (Yes)	Enrolment: Cohort = chosen cohort, Student = <code>{!Student_Lookup.recordId}</code> , Status = Confirmed
Screen (Yes)	Success confirmation showing the new record
Screen (No)	“Sorry — that cohort just filled up”, with Back
Fault paths	Every Get/Create/Action element gets a Fault connector to an error screen showing <code>{!\$Flow.FaultMessage}</code>

Save as “Quick Enrol Student”, Activate. Surface it via App Launcher or a Quick Enrol button on the Cohort page.

6.3 Scheduled flow — Cohort_Start_Reminder (US-09)

1. New Flow → **Schedule-Triggered Flow** · Frequency **Daily** at 07:00.
2. **Get Records**: all Enrolments where `Status__c = Confirmed`.
3. **Loop** over them → **Decision**: `{!Loop.Cohort__r.Start_Date__c} = {!fReminderDate}`, where `fReminderDate` is a Date formula `{!$Flow.CurrentDate} + 3`.
4. On Yes → **Assignment**: build a Task (Subject “Reminder: your cohort starts in 3 days”, WhatId = enrolment Id, ActivityDate today, Priority High) and add it to a Task collection.
5. After the loop → Decision “any tasks?” → **Create Records** from the collection (one DML, outside the loop — the bulkification pattern in Flow).
6. Save as “Cohort Start Reminder”, Activate.

7 · Apex (Lab 4)

Create each class in VS Code (`force-app/main/default/classes`) or Developer Console (gear icon → Developer Console → File → New → Apex Class). Full source is in the GitHub repo — the design is summarised here.

7.1 EnrolmentTrigger — routing only

```
trigger EnrolmentTrigger on Enrolment__c (before insert, before update, after insert, after update) {
    if (Trigger.isBefore) {
        if (Trigger.isInsert)      { EnrolmentTriggerHandler.onBeforeInsert(Trigger.new); }
        else if (Trigger.isUpdate) { EnrolmentTriggerHandler.onBeforeUpdate(Trigger.new, Trigger.oldMap); }
    }
    if (Trigger.isAfter && (Trigger.isInsert || Trigger.isUpdate)) {
        EnrolmentTriggerHandler.onAfterChange(Trigger.new);
    }
}
```

7.2 EnrolmentTriggerHandler — all the logic

Method	Story	What it does
<code>setDefaults</code>	US-02	Defaults Enrolment Date to today, Status to Applied, Amount Paid to 0.
<code>buildUniqueKey</code>	US-05	Writes <code>Enrolment_Key__c = CohortId-StudentId</code> ; the unique external ID makes the database itself reject duplicates.
<code>preventOverbooking</code>	US-04	Bulk-safe capacity rule: collect cohort IDs → one SOQL query → allocate seats in memory → <code>addError()</code> on records that don't fit.
<code>enqueueStudentSummary</code>	US-13	Hands off to Queueable Apex after commit.

7.3 StudentSummaryQueueable

Aggregates each student's non-cancelled enrolments with one `GROUP BY` query and writes `Contact.Total_Enrolments__c` . Asynchronous = fresh governor limits, runs after the transaction commits.

7.4 CohortAvailabilityService & EnrolmentStatusService — the AI bridge

Both are `@InvocableMethod` classes: one method, three consumers (screen flow, Agentforce agent, REST). `findAvailableCohorts` returns open future cohorts with seats for a course-name search; `checkStatus` returns a student's enrolments by email. Every input/output has a clear `@InvocableVariable` description — the agent reads these to decide what to pass.

7.5 CohortController & EnrolmentController — the LWC layer

`@AuraEnabled(cacheable=true)` for reads (enables `@wire`), plain `@AuraEnabled` for the write. The write wraps DML in try/catch and rethrows `AuraHandledException` with a friendly sentence (NFR-04) — including translating the duplicate-key error into “That student is already enrolled on this cohort.”

Checkpoint (US-04, US-05): try to enrol a 4th student onto CO-0001 (capacity 3) — the save must fail with the “cohort is full” message. Enrol the same student twice — that must fail too.

8 · Lightning Web Components + Enrolment Console (Lab 5)

An LWC is three files: HTML template, JavaScript class, and a meta XML that says where it may be used. Create them in VS Code under `force-app/main/default/lwc` (right-click → SFDX: Create Lightning Web Component).

Component	Role	Key techniques
<code>cohortExplorer</code>	Course combobox + datatable of open cohorts	<code>@wire</code> to both controller reads; row selection dispatches a <code>cohortselected</code> CustomEvent; public <code>@api refresh()</code> calls <code>refreshApex</code>
<code>enrolmentPanel</code>	Contact picker + Enrol button	Receives cohort via <code>@api</code> ; imperative Apex call; <code>ShowToastEvent</code> ; <code>NavigationMixin</code> to the new record; fires <code>enrolled</code> event
<code>enrolmentConsole</code>	Wrapper wiring the two together	Data down (<code>cohort={selectedCohort}</code>), events up (<code>oncohortselected</code> , <code>onenrolled</code> → <code>explorer.refresh()</code>)

8.1 Create and activate the app page (manual step)

1. Setup → **Lightning App Builder** → the deployed page **Enrolment Console** already exists (or New → App Page → one region → drop **Enrolment Console** component on it).
2. Open it → **Activation...** → Lightning Experience tab → add to an app (e.g. Sales) → Save.
3. App Launcher → “Enrolment Console” → filter by course, select a cohort, pick a student, click **Enrol student**.

Checkpoint (US-10): a toast appears, the seat count drops after refresh, and an overbooking attempt is still blocked by the Week-2 trigger — the console is just another channel over the same protected service layer.

9 · Agentforce agent (Lab 6 — manual, in the org)

Agent metadata is not part of the CLI deployment — build it in Setup. If Agentforce is unavailable in your Developer org, the two invocable Apex actions are still deployed and the learning objective (the `@InvocableMethod` bridge) survives.

1. **Enable:** Setup → **Einstein Setup** → turn on Einstein → then Setup → **Agentforce Agents** (assign yourself the Agentforce permission set if prompted).
2. **Create the agent:** New Agent → type Employee/internal assistant · Name **Oxfordable Student Assistant** · Role: “Answers questions about our courses, cohort availability and a student’s own enrolment status.” · Company: “Oxfordable Careers, a technology training provider.” · Tone: warm, concise, professional — never invent dates, prices or seat numbers.
3. **Create two actions** (Agent Actions → New → Apex):

Action	Apex	Input	Output
Find Available Cohorts	CohortAvailabilityService.findAvailableCohorts	Course Name (text)	Answer, Available Cohorts, Seats Available
Check Enrolment Status	EnrolmentStatusService.checkStatus	Student Email (text)	Answer, Found

Write every description **for a colleague who has never seen your org** — thin descriptions are the number-one reason agents misbehave.

4. **Create the topic** “Course Enrolment Enquiries”: classification = “Use this topic when someone asks about our courses, when a cohort starts, whether seats are available, how to join, or the status of an enrolment they already hold.” Scope: answer only from the assigned actions. Instructions: confirm the course before searching · always state start date and seats remaining · offer the waiting list when full · ask for the booking email for status questions · never share one student’s details with another · pass anything else to the team. Assign both actions.
5. **Test in the conversation preview:** “Do you have any Advanced cohorts with space?” → live list · “What is the status of my enrolment?” → asks for email · an off-topic question → graceful decline. After each test open the **reasoning trace** to see which topic and action fired.
6. **Activate** the agent.

10 · Testing (Lab 7)

Three test classes are deployed (all green in the target org):

Class	Covers
TestDataFactory	Reusable course/cohort/student/enrolment creation. Contacts are inserted with <code>DuplicateRuleHeader.allowSave = true</code> so org duplicate rules never break the tests. No hard-coded IDs, no org data.
EnrolmentTriggerTest	Defaults (US-02) · overbooking blocked (US-04, negative test with <code>Database.insert(rec, false)</code>) · 200-record bulk test (NFR-01) asserting the capacity ceiling is never exceeded · duplicate blocked (US-05) · Queueable updates Contact (US-13, forced by <code>Test.stopTest()</code>)
CohortServicesTest	Both controllers, both invocable services, happy and error paths

```
sf apex run test --target-org <alias> --code-coverage --result-format human --wait 10
```

Two real-world lessons baked into the tests: (1) WITH `SECURITY_ENFORCED` means the running user needs FLS — assign the permission set before running tests as yourself. (2) A 200-record partial-save with mixed successes triggers Salesforce retry behaviour; the bulk test asserts the true business invariant — the cohort can *never* be oversold.

11 · Security & deployment (Lab 8)

11.1 Permission set (NFR-05 — never edit profiles)

The deployed **Enrolment Console User** permission set grants: Read/Create/Edit on Course, Cohort, Enrolment · Read on Contact · FLS on all custom fields · access to the four Apex classes and the Quick Enrol flow · the three tabs visible. Assign it:

```
sf org assign permset --name Enrolment_Console_User --target-org <alias>
```

11.2 Version control

```
git clone https://github.com/dangutdavid/oxfodablenameveloper1_training.git
cd oxfodablenameveloper1_training
git checkout -b feature/my-change # branch, change, commit, push, PR
```

11.3 Deploy to any org

```
# authorise
sf org login web --alias targetorg
# validate first - runs every check, saves nothing
sf project deploy start --source-dir force-app --target-org targetorg --test-level RunLocalTests --dry-run
# then deploy for real
sf project deploy start --source-dir force-app --target-org targetorg --test-level RunLocalTests
```

11.4 Post-deployment checklist

1. Confirm all three Flows are **Active** (Setup → Flows).
2. Assign the **Enrolment Console User** permission set to users.
3. Activate the **Enrolment Console** page and add tabs to app navigation (Section 8.1).
4. Build/activate the Agentforce agent and re-test one question (Section 9).
5. Deliverability = **All Email**.
6. Load reference data — **records are not deployed**: `sf apex run --file scripts/apex/seed.apex --target-org targetorg`.
7. Run smoke tests UAT-03, UAT-04, UAT-10 below.
8. Tag the release: `git tag v1.0 && git push --tags`.

12 · UAT script — prove it works

ID	Story	Test step	Expected result
UAT-01	US-01	Create a Course and a Cohort linked to it	Both save; the Cohort shows the Course
UAT-02	US-01	Set Cohort End Date before Start Date	Validation error shown
UAT-03	US-03	Create one enrolment on a capacity-3 cohort	Enrolled Count = 1, Seats Remaining = 2
UAT-04	US-04	Enrol a 4th student on a capacity-3 cohort	Save blocked with the custom capacity message
UAT-05	US-05	Enrol the same student twice on one cohort	Save blocked (duplicate key)
UAT-06	US-06	Change an enrolment Status to Confirmed	Email sent to the student; follow-up Task created
UAT-07	US-07	Fill the final seat on a cohort	Cohort Status becomes Full automatically
UAT-08	US-08	Run the Quick Enrol screen flow end to end	Enrolment created; confirmation screen shown
UAT-09	US-09	Cohort start date = today+3; run the scheduled flow	Reminder Task for confirmed enrolments only
UAT-10	US-10	Enrolment Console: filter, select, enrol	Toast shown; seat count refreshes; record created
UAT-11	US-11	Ask the agent “which Advanced cohorts have seats?”	Live cohorts with dates and seats remaining
UAT-12	US-12	Ask the agent for status with a known email	Cohort, start date and status returned
UAT-13	US-13	Check the student’s Contact record	Total Enrolments reflects active enrolments
UAT-14	US-14	Run all Apex tests	All pass; coverage ≥ 75%
UAT-15	US-14	Deploy to a second org; repeat UAT-03 & UAT-10	Both behave identically

Appendix · What is deployed where

Type	Components
Custom Objects	Course__c, Cohort__c, Enrolment__c (+ Contact.Total_Enrolments__c)
Validation Rules	End_Date_After_Start_Date, Payment_Not_Above_Fee
Flows (Active)	Enrolment_Confirmed_Actions, Quick_Enrol_Student, Cohort_Start_Reminder
Apex	EnrolmentTrigger, EnrolmentTriggerHandler, StudentSummaryQueueable, CohortAvailabilityService, EnrolmentStatusService, CohortController, EnrolmentController, TestDataFactory, EnrolmentTriggerTest, CohortServicesTest
LWC	cohortExplorer, enrolmentPanel, enrolmentConsole
Pages / Tabs	Enrolment_Console app page · Course, Cohort, Enrolment tabs
Security	Enrolment_Console_User permission set (assigned to maren.dangut@gmail.com)
Scripts	scripts/apex/seed.apex · manifest/package.xml

Appendix B · Exact reproduction — every command used

This is the complete, ordered command sequence used to build, test, publish and deploy this project. Replace `lse-demo` with your own org alias and the GitHub URL with your own repository to reproduce the whole exercise end to end.

B.1 Environment checks

```
sf --version           # confirm the Salesforce CLI is installed
git --version          # confirm Git
gh auth status         # confirm GitHub CLI is authenticated (or use git credentials)
sf org list            # confirm the target org is authorised
# if the org is not yet authorised:
sf org login web --alias lse-demo
```

B.2 Create the project and write the source

```
cd "<your working folder>"
sf project generate --name careerlaunch --template standard
cd careerlaunch
```

Then create every file under `force-app/main/default` exactly as in the GitHub repo:

Folder	Files
objects/	Course__c, Cohort__c, Enrolment__c (object + fields + validationRules folders), Contact/fields/Total_Enrolments__c
tabs/	Course__c, Cohort__c, Enrolment__c tab-meta.xml
classes/	9 Apex classes + their .cls-meta.xml files
triggers/	EnrolmentTrigger.trigger + meta
lwc/	cohortExplorer, enrolmentPanel, enrolmentConsole (html + js + js-meta.xml each)
flows/	Enrolment_Confirmed_Actions, Quick_Enrol_Student, Cohort_Start_Reminder flow-meta.xml
flexipages/	Enrolment_Console.flexipage-meta.xml
permissionsets/	Enrolment_Console_User.permissionset-meta.xml
manifest/ + scripts/apex/	package.xml · seed.apex

Fastest route: clone the finished repo instead of typing — `git clone https://github.com/dangutdavid/oxfodabledeveloper1_training.git`

B.3 Deploy to the org

```
# full deploy of all 49 components
sf project deploy start --source-dir force-app --target-org lse-demo --wait 15

# assign the permission set to yourself (required BEFORE running tests -
# WITH SECURITY_ENFORCED needs field-level security on the new fields)
sf org assign permset --name Enrolment_Console_User --target-org lse-demo
```

B.4 Run the tests

```
sf apex run test --target-org lse-demo \  
  --tests EnrolmentTriggerTest --tests CohortServicesTest \  
  --code-coverage --result-format human --wait 10  
# expected: 10/10 Pass, class coverage 79-100%
```

B.5 Seed the sample data

```
sf apex run --file scripts/apex/seed.apex --target-org lse-demo  
# creates 2 courses, 2 open cohorts (capacity 3 and 10), 3 student contacts
```

B.6 Publish to GitHub

```
git init  
git add .  
git commit -m "CareerLaunch v1: data model, automation, Apex, LWC, flows, permission set"  
git branch -M main  
git remote add origin https://github.com/dangutdavid/oxfodababledeveloper1_training.git  
git push -u origin main
```

B.7 Useful everyday commands

```
sf org open --target-org lse-demo # open the org in a browser  
sf project retrieve start --manifest manifest/package.xml \  
  --target-org lse-demo # pull org changes back to source  
sf project deploy start --source-dir force-app/main/default/classes \  
  --target-org lse-demo # redeploy just the Apex  
sf apex run --file scripts/apex/seed.apex --target-org lse-demo # run anonymous Apex  
sf apex tail log --target-org lse-demo # stream debug logs  
sf project deploy start --source-dir force-app --target-org <other> \  
  --test-level RunLocalTests --dry-run # validate-only against a 2nd org
```

B.8 The original build prompt (AI-assisted reproduction)

This project was built with an AI coding assistant (Claude Code) from the three participant workbooks and the programme handbook. The exact instruction given was:

```
i want you to read this attached materials - create salesforce project and write all  
the code - creating the objects and fields needed, validation rules, pages, flows,  
apex, agents, etc  
commit and push to  
https://github.com/dangutdavid/oxfodababledeveloper1_training  
deploy to salesforce org username - maren.dangut@gmail.com  
after that create a pdf document with step by step for user to follow to create each.
```

To reproduce with your own AI assistant: attach the Week 1-3 participant workbook PDFs and the 3-Week Programme Handbook, swap in your own repo URL and org username, and use the same prompt. Review every generated file against Sections 2-11 of this guide before deploying.

B.9 Fixes discovered during the real deployment (learn from these)

Symptom	Root cause	Fix applied
Contact inserts failed in tests with <code>DUPLICATES_DETECTED</code>	The org has an active Contact duplicate rule	TestDataFactory inserts contacts with <code>Database.DMLOptions.DuplicateRuleHeader.allowSave = true</code>
QueryException "secure query included inaccessible field"	WITH <code>SECURITY_ENFORCED</code> enforces the running user's FLS; API-created fields grant FLS to nobody	Assign the Enrolment_Console_User permission set before running tests
<code>CANNOT_EXECUTE_FLOW_TRIGGER</code> on confirmed enrolments in tests	The flow's Send Email action failed (deliverability/limits) and killed the save	Fault path on the email action so a failed email never blocks the enrolment
Bulk test expected exactly 150 successes, got fewer	Partial-save retries + the unique key make "exactly capacity" non-deterministic in one 200-record batch	Assert the true invariant instead: the cohort can never exceed capacity, and the 50 overflow records are always blocked